```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, when
```

```

spark = SparkSession.builder \
    .appName("Unclean Financial Data Analysis") \
    .getOrCreate()
```

```
# 2. Read the unclean dataset
df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv("/content/unclean_financial_data_extended_275k.csv")
```

```
total_rows = df.count()
total_rows
```

```
275000
```

```
from pyspark.sql.functions import col, count, when, isnan

# Count nulls or NaNs in each column
null_counts = df.select([
    count(when(col(c).isNull() | isnan(c), c)).alias(c)
    for c in df.columns
])

# Show result as a single row (each column = null count)
null_counts.show(truncate=False)
```

```
+------+----+----------+----------+---------+----+--------------+---------+---------+---------+----------+-------------+---------+------
|Income|Age |Dependents|Occupation|City_Tier|Rent|Loan_Repayment|Insurance|Groceries|Transport|Eating_Out|Entertainment|Utilities|Health
+------+----+----------+----------+---------+----+--------------+---------+---------+---------+----------+-------------+---------+------
|5454  |5420|0         |0         |0        |0   |0             |0        |5428     |0        |0         |5406         |0        |0
+------+----+----------+----------+---------+----+--------------+---------+---------+---------+----------+-------------+---------+------
```

```
# 3. Drop the extra columns
cols_to_drop = ["random_note", "temp_value"]
df = df.drop(*cols_to_drop)
```

```

# 4. Remove any rows with null values
df = df.na.drop()

```

```
# 5. Reorder columns to match the original schema
ordered_cols = [
    "Income", "Age", "Dependents", "Occupation", "City_Tier",
    "Rent", "Loan_Repayment", "Insurance", "Groceries", "Transport",
    "Eating_Out", "Entertainment", "Utilities", "Healthcare", "Education",
    "Miscellaneous", "Desired_Savings_Percentage", "Desired_Savings",
    "Disposable_Income", "Potential_Savings_Groceries",
    "Potential_Savings_Transport", "Potential_Savings_Eating_Out",
    "Potential_Savings_Entertainment", "Potential_Savings_Utilities",
    "Potential_Savings_Healthcare", "Potential_Savings_Education",
    "Potential_Savings_Miscellaneous"
]
df_clean = df.select(*ordered_cols)

```

```
# 6. (Optional) Write the cleaned DataFrame back out as CSV
# To save as a single CSV file, you can repartition the DataFrame to 1
df_clean.repartition(1).write \
    .option("header", True) \
    .mode("overwrite") \
    .csv("/mnt/data/Project_DataSet.csv")
```

```
df_clean.count()
```

```
255381
```

```
df = df.filter(df["Income"].isNotNull())

```

```
# Count nulls or NaNs in each column
null_counts = df.select([
    count(when(col(c).isNull() | isnan(c), c)).alias(c)
    for c in df.columns
])

# Show result as a single row (each column = null count)
null_counts.show(truncate=False)
```

```
+------+---+----------+----------+---------+----+--------------+---------+---------+---------+----------+-------------+---------+-------
|Income|Age|Dependents|Occupation|City_Tier|Rent|Loan_Repayment|Insurance|Groceries|Transport|Eating_Out|Entertainment|Utilities|Healthc
+------+---+----------+----------+---------+----+--------------+---------+---------+---------+----------+-------------+---------+-------
|0     |0  |0         |0         |0        |0   |0             |0        |0        |0        |0         |0            |0        |0
+------+---+----------+----------+---------+----+--------------+---------+---------+---------+----------+-------------+---------+-------
```

```
df.count()
```

```
255381
```